

Relatório sobre a atividade “25 - Prática: Git e Github para Iniciantes (III)”

Lucas Gabriel Arenhardt

1. Introdução

O foco deste módulo foi entender o que é e como usar o Git e Github. O curso da Udemy, intitulado “Git e Github para iniciantes”, ensinou a utilizar o Git e Github no terminal, enquanto o vídeo do Youtube “Como usar o Github Desktop” mostrou sua utilização através de uma interface gráfica.

2. Desenvolvimento

2.1. O que é Git e Github?

Git é um sistema de controle de versões distribuído que permite aos desenvolvedores rastrear mudanças no código ao longo do tempo. Ele facilita o trabalho em equipe, permitindo que múltiplas pessoas colaborem no mesmo projeto sem conflitos, armazenando o histórico completo das alterações feitas. Isso ajuda a gerenciar diferentes versões de um projeto e reverter mudanças, se necessário.

GitHub, por sua vez, é uma plataforma online que hospeda repositórios Git. Ele facilita a colaboração em projetos ao fornecer ferramentas para compartilhar código, revisar alterações, abrir issues (problemas ou sugestões) e integrar com outras ferramentas de desenvolvimento. GitHub também oferece funcionalidades como integração contínua, automação de processos e o uso de pull requests para gerenciar mudanças colaborativas no código.

2.2. Configurando o Git

Para configurar o Git numa máquina, deve-se inserir os seguintes comandos no terminal:

- `git config --global user.name [nome desejado]`
- `git config --global user.email [email desejado]`

2.3. Iniciando um repositório

Para criar um repositório, primeiro deve-se navegar até a pasta desejada. Em seguida, basta inserir o comando “git init”.

2.4. Ciclo de vida dos arquivos no Git

Quando um arquivo é criado ou adicionado ao repositório pela primeira vez, ele está no estado “untracked” - isso significa que o Git ainda não está o monitorando. Para começar a rastreá-lo, o arquivo precisa ser adicionado a “staging area” usando comando “git add [nome do arquivo]”.

Esse arquivo na “staging area” é, então, rotulado como “staged”, indicando que as mudanças estão prontas para serem incluídas no próximo commit.

Ao modificar um arquivo rastreado, ele entra no estado “modified”. Isso indica que o arquivo foi alterado, mas as mudanças ainda não foram preparadas para o commit.

Depois de adicionar o arquivo a um repositório e fazer um commit, o arquivo entra no estado “unmodified”, indicando que o conteúdo do arquivo no diretório de trabalho é idêntico ao que foi salvo no último commit.

2.5. Logs

Os logs no Git são um registro detalhado de todas as mudanças realizadas no repositório ao longo do tempo. Eles exibem o histórico de commits feitos no repositório, permitindo que se visualize quais alterações foram feitas, quando, por quem, e com mensagens associadas a cada commit. O comando mais básico para visualizar os logs é “git log”. Também é possível utilizar “git shortlog” para uma visão mais resumida, ou até filtrar os logs por autor utilizando “git log --author=[nome]”.

2.6. Diferenças

O comando “git diff” pode ser utilizado para verificar o que foi alterado antes de realizar um commit ou para comparar o histórico de commits anteriores.

2.7. Desfazendo modificações

Desfazer commits no Git é uma tarefa comum, especialmente quando algo foi cometido incorretamente, e pode ser feito de várias maneiras, dependendo do que se deseja corrigir.

Um dos métodos é utilizar o comando “git checkout [nome do arquivo]”. Esse comando é utilizado para reverter a modificação feita em um arquivo que ainda não está na “staging area”. Já caso o objeto já esteja na situação “staged”, deve-se usar o comando “git reset HEAD nome”.

Para os arquivos que já receberam commit, há três variações possíveis para desfazê-los. O comando utilizado é “git reset [parâmetro]”, onde o parâmetro pode ser ‘soft’, ‘mixed’ ou ‘hard’. O ‘soft’ desfaz o commit, mas mantém as mudanças na área de stage. ‘Mixed’ desfaz o commit e remove as mudanças da área de stage, mas mantém as alterações no diretório de trabalho. Já o ‘hard’ desfaz o commit e remove todas as mudanças tanto da área de stage quanto do diretório de trabalho, perdendo permanentemente as alterações.

2.8. Criando um repositório no GitHub

Para criar um repositório no GitHub, basta ir no site e encontrar a aba “Criar um novo repositório”. Deve-se, então, escolher um nome para o repositório, opcionalmente dar uma descrição, e escolher se ele será público ou privado.

2.9. Ligando um repositório local a um remoto

Para realizar o vínculo entre um repositório local e um remoto, basta utilizar os seguintes comandos (após a configuração da chave SSH):

- git remote add origin
- git push -u origin

2.10. Enviando mudanças a um repositório remoto

Para enviar as mudanças feitas no git a um repositório remoto, é necessário inserir o seguinte comando: “git push origin master”

2.11. Clonando um repositório

Clonar um repositório serve para copiar um repositório Git remoto para a máquina local. Isso cria uma cópia completa do histórico e dos arquivos do repositório, permitindo que se trabalhe no projeto de forma independente.

Para clonar um repositório, basta utilizar o comando “git clone [endereço do repositório] [novo nome]”.

2.12. Fork

O fork é uma operação realizada no GitHub, que cria uma cópia independente de um repositório de código sob a conta do usuário. Isso permite que o usuário faça alterações no código sem afetar o repositório original, facilitando contribuições a projetos de código aberto ou a personalização de projetos existentes. Após o fork, o usuário pode modificar seu repositório de forma autônoma e, se desejar, propor essas alterações de volta ao repositório original por meio de um pull request.

2.13. Branch

Uma branch (ou ramificação) em Git é uma linha independente de desenvolvimento dentro de um repositório, permitindo que se trabalhe em diferentes versões do projeto simultaneamente. Cada branch pode conter alterações e commits distintos sem interferir na branch principal (geralmente chamada de main ou master). Isso é útil para desenvolver novas funcionalidades, corrigir bugs ou experimentar mudanças sem afetar a estabilidade do código principal. Alguns comandos úteis são:

- **Criar uma branch:** git checkout -b [nome da branch]
- **Visualizar branches:** git branch
- **Trocar entre branches:** git checkout [nome da branch]
- **Apagar uma branch:** git branch -D [nome da branch]

2.13.1. Mesclar Branches

Mesclar (ou merge) no Git é o processo de combinar alterações de diferentes branches em um único branch. Isso é fundamental para integrar o trabalho de várias pessoas ou diferentes partes de um projeto. Há duas maneiras diferentes de realizar essa operação: utilizando merge ou rebase.

- **Merge:** (`git merge [nome da branch]`) é uma operação não destrutiva, onde é gerado um commit extra para apontar para a bifurcação gerada. Seu uso acarreta em um histórico poluído;
- **Rebase:** (`git rebase [nome da branch]`) ao invés de criar um commit extra, o rebase coloca o commit da branch no fim da lista, mantendo o histórico linear. O ponto ruim é que perde-se a ordem cronológica.

2.14. Gitignore

O `.gitignore` é um arquivo de configuração no Git que define quais arquivos e diretórios devem ser ignorados pelo sistema de controle de versão. Isso significa que qualquer item listado no `.gitignore` não será rastreado, nem incluído em commits. O arquivo é usado para evitar o versionamento de arquivos desnecessários ou específicos do ambiente de desenvolvimento, como arquivos temporários, logs, e configurações locais.

2.15. Git stash

O comando `git stash` é usado para salvar temporariamente mudanças não comprometidas (modificações no diretório de trabalho e na área de stage) e restaurar o repositório ao estado do último commit. Isso é útil quando se deseja interromper o trabalho atual para lidar com outra tarefa ou mudar de branch, mas sem perder ou comprometer as alterações em andamento.

2.16. Alias

Um alias é uma forma de criar atalhos personalizados para comandos Git, simplificando o uso de comandos longos e complexos. Ao definir um alias, você pode associar um comando Git a um nome mais curto ou mais intuitivo, economizando tempo e tornando o fluxo de trabalho mais eficiente. Por exemplo, ao criar um alias para `git status`, você poderia usar `git st` para o mesmo comando.

Para fazer isso, basta utilizar o comando `“git config –global alias.[novo comando] [comando]”`.

2.17. Tags

No Git, uma tag é uma referência a um ponto específico na história do repositório, geralmente utilizada para marcar versões importantes, como lançamentos de software. Tags são como "marcadores" que apontam para commits específicos, facilitando a identificação de versões estáveis ou marcos significativos no desenvolvimento.

2.18. GitHub Desktop

O GitHub Desktop é uma aplicação gráfica desenvolvida pelo GitHub que simplifica o uso do Git e GitHub para usuários que preferem uma interface visual em vez de trabalhar com linha de comando. Com o GitHub Desktop, é possível realizar operações comuns de controle de versão e gerenciamento de repositórios de forma intuitiva, sem precisar digitar comandos.

Ele oferece uma interface amigável e simples, que permite visualizar e gerenciar mudanças de código, commits, branches e tags. Também facilita a sincronização com repositórios do GitHub, permitindo fazer push e pull de alterações, gerenciar pull requests e visualizar problemas diretamente na aplicação.

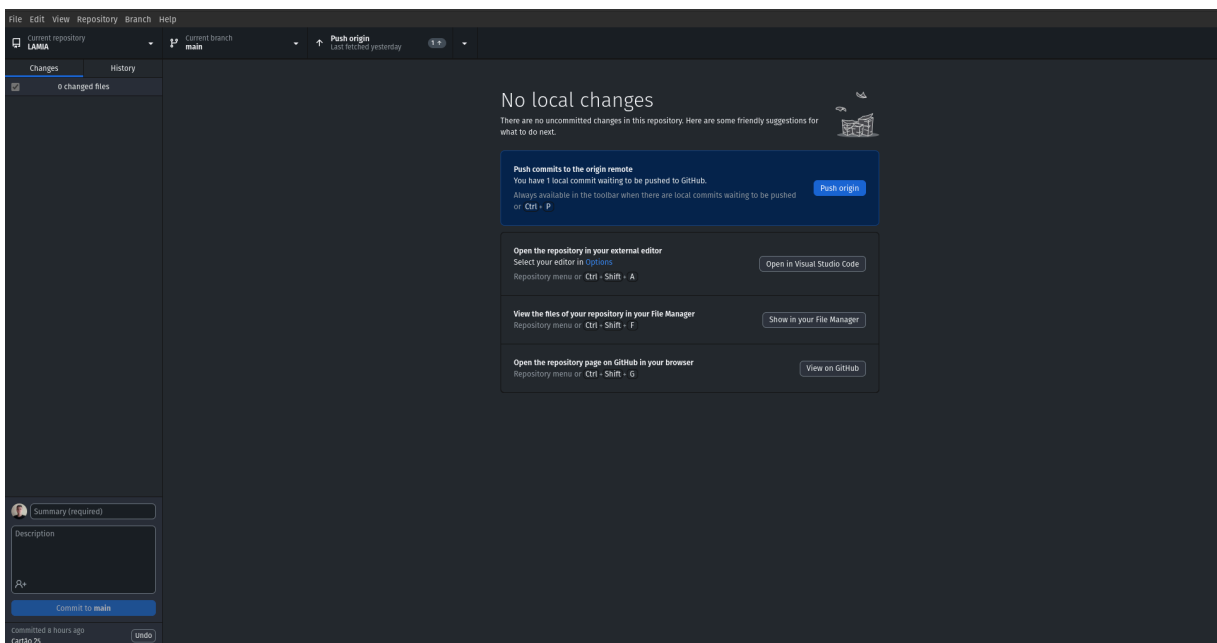


Imagem 1 - Interface do GitHub Desktop.

4. Conclusão

Este módulo abordou conceitos essenciais do Git e GitHub, destacando suas funcionalidades e o papel crucial que desempenham no desenvolvimento de software moderno. A compreensão e o uso eficaz dessas ferramentas são fundamentais para o controle de versões e a colaboração em projetos de programação.

O Git permite um gerenciamento detalhado das mudanças no código, enquanto o GitHub proporciona uma plataforma robusta para hospedagem e colaboração em projetos. As habilidades adquiridas, como configurar o Git, criar e gerenciar repositórios, utilizar branches e realizar merges, são práticas indispensáveis para manter um fluxo de trabalho eficiente e colaborativo. A adição de conhecimento sobre ferramentas como GitHub Desktop e comandos avançados como alias e stash fortalece a capacidade de gerenciar projetos complexos e otimizar o processo de desenvolvimento.

O domínio desses conceitos e práticas não apenas melhora a eficiência do desenvolvimento de software, mas também prepara para lidar com projetos em equipe.